01 Introduction to Random Forest



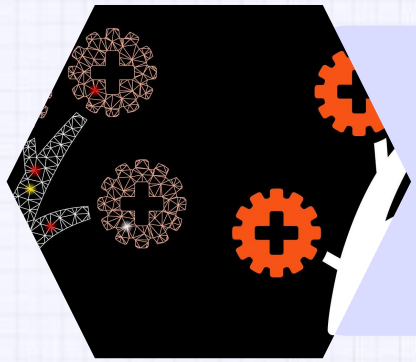
Definition and Basic Concept

The representative method of ensemble learning: Random forest significantly improves the accuracy and generalization ability of the model by constructing multiple decision trees and integrating their prediction results, making it a powerful tool for solving classification and regression problems.

Dual introduction of randomness: By randomly selecting samples (Bootstrap sampling) and features (feature subset partitioning), the risk of model overfitting is effectively reduced and diversity is enhanced.

Wide range of application scenarios: Due to its robustness and ease of use, random forests are widely used in financial risk control, medical diagnosis, image recognition, and other fields, especially performing well in processing high-dimensional data.

Key Components: Decision Trees

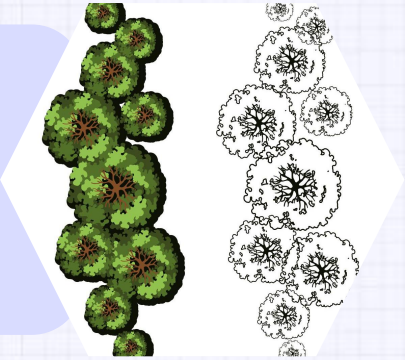


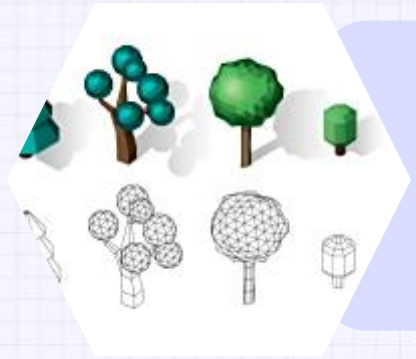
Splitting criteria

Gini Index or Information Gain are commonly used as the basis for node splitting, and the optimal features are selected to partition the data.

Pruning optimization

Control model complexity by pre pruning (limiting tree depth and leaf node sample size) or post pruning (cost complexity pruning), balancing bias and variance.

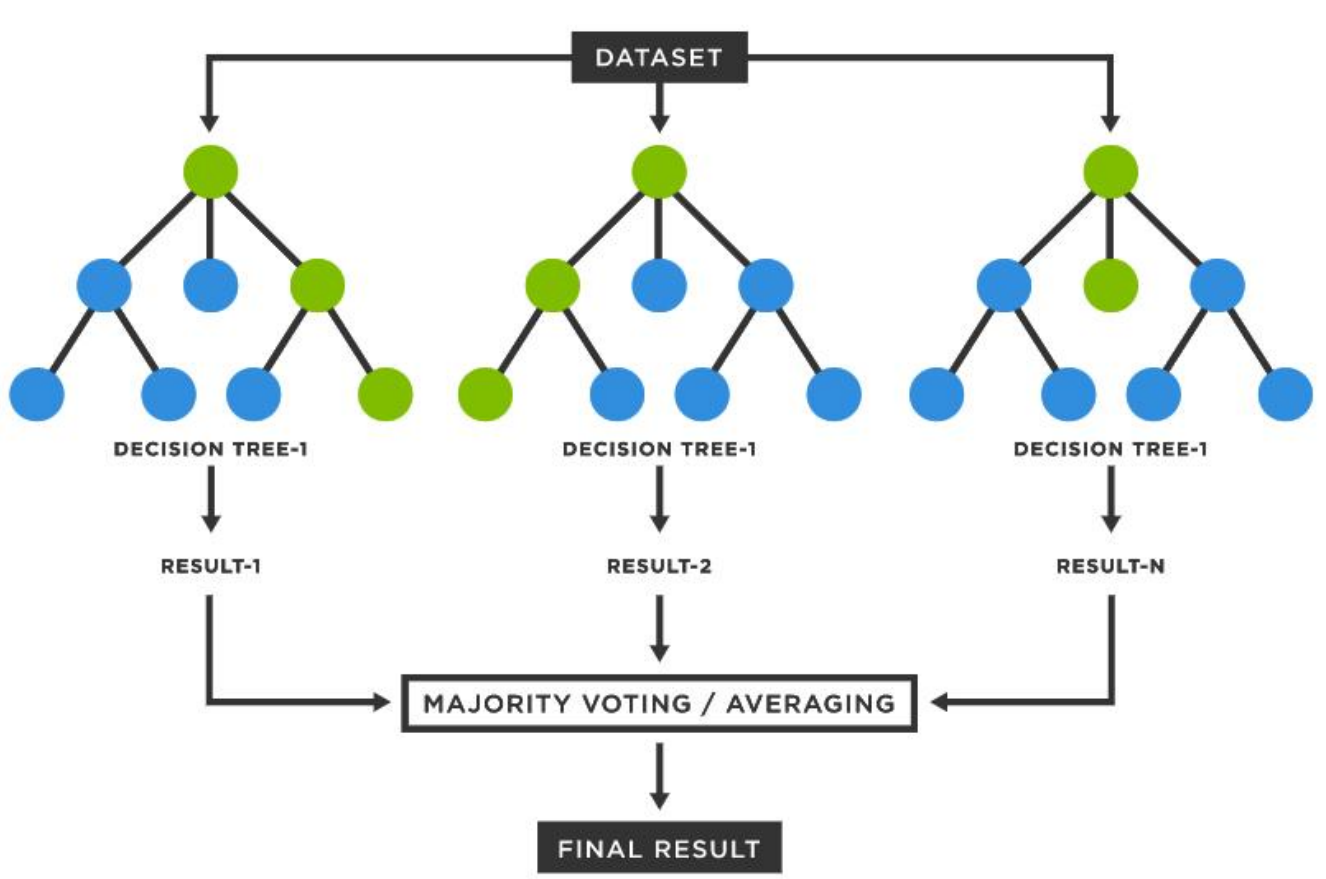




Parallel training

The construction of each decision tree is independent of each other, supports parallel computing, greatly improves training efficiency, and is suitable for large-scale datasets.

Ensemble Learning Overview



Bagging (Bootstrap Aggregating) generates multiple training subsets through replacement sampling, trains the base learner separately, and then votes to integrate them. Random forest is an extension of Bagging, which introduces additional feature randomness.

Ensemble Learning Overview

Compared with ordinary bagging, random forest has lower generalization error, further reducing inter tree correlation and **enhancing overall stability due to feature randomness**.

The classification task **uses the Majority Voting** method, while the regression task uses the Average method to reduce the interference of individual noisy data through democratic decision-making.

Support **output probability estimation (Soft Voting),** providing flexibility for subsequent model fusion or threshold adjustment.

02 How Random Forest Works



Bagging (Bootstrap Aggregating)

- Self sampling training: Each decision tree uses bootstrap sampling to extract an equal amount of samples from the original training set (such as 60 out of 60 samples, allowing for repetition), ensuring the diversity of training data for each tree. For example, a sample may be selected multiple times, and its weight will correspondingly increase in the model.

- Parallel independent training: All decision trees are trained in parallel based on different subsets of data, without any dependency on each other. This independence improves the model's generalization ability by reducing variance, especially for high variance and low bias base learners such as deep decision trees.

- Aggregate prediction results: Majority voting is used in classification tasks, while mean output is used in regression tasks. For example, if 5 trees predict category A and 3 trees predict category B, the final result will be A, effectively reducing the risk of overfitting.

Feature Randomness

- Random feature subset: When splitting nodes, only a subset is randomly selected from all features (such as $\sqrt{p}$ features, where p is the total number), breaking the correlation between features. For example, in image classification, one tree may split based solely on color features, while another tree enhances diversity based on texture.

- Reduce overfitting: Force each tree to focus on different feature combinations, avoid a few strong features dominating the global, and improve model robustness. Experiments have shown that feature randomness can maintain high accuracy of forests in noisy data.

- Processing high-dimensional data: For scenarios where the number of features far exceeds the sample size (such as gene expression data), random selection significantly reduces computational overhead while preserving key feature information.

- Synergistic effect with Bagging: The double randomness of data (samples) and features forms a "double random forest", which further reduces model variance and is superior to a single randomization strategy.

Voting Mechanism for Classification



Majority voting principle

Each decision tree independently predicts category labels, and the final result is determined by the category with the highest number of votes. For example, in medical diagnosis, if 70% of the trees are judged as "diseased", a positive diagnosis is output.



Dealing with tie votes

When multiple categories have the same number of votes, it can be resolved through preset rules (such as prioritizing the category with a higher probability estimate) or introducing soft voting (weighted probability averaging).



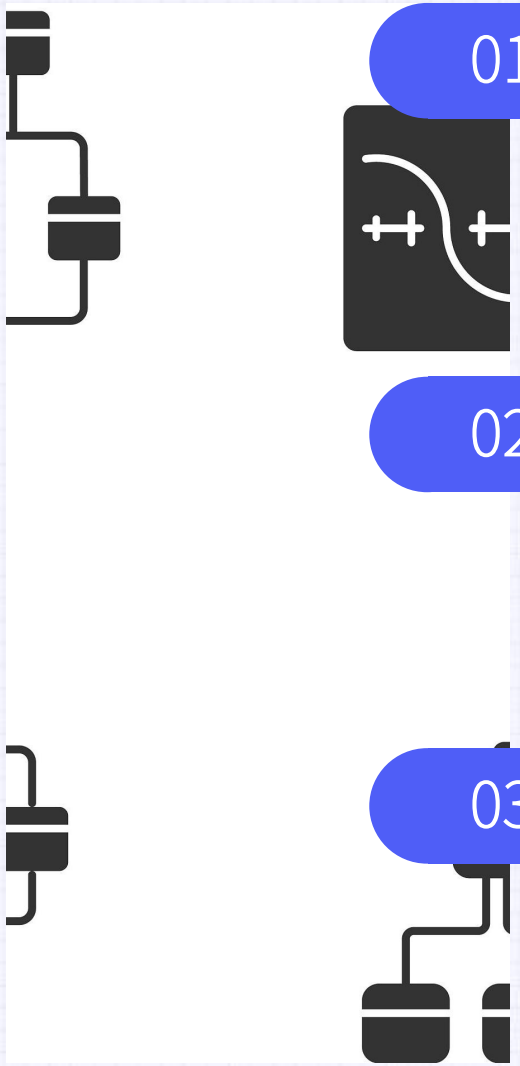
Probability calibration capability

By calculating the category voting ratio of each tree, probability estimates (such as "80% confidence") can be output, which is suitable for scenarios that require uncertainty quantification (such as financial risk assessment).

03 Advantages of Random Forest



High Accuracy and Robustness



01

Advantages of ensemble learning

Random forests significantly reduce the risk of overfitting that may occur with a single decision tree by constructing multiple decision trees and synthesizing their prediction results, thereby improving the overall accuracy and generalization ability of the model.

02

Anti noise capability

Due to the use of bootstrap and random feature selection in random forests, the model has a high tolerance for noise and outliers in the training data, and can maintain stable predictive performance.

03

Suitable for complex data

Whether it is a linearly separable or non linearly separable dataset, random forests can effectively capture complex patterns in the data through the voting mechanism of multiple trees, making them suitable for classification and regression tasks.

Handling High-Dimensional Data

- Automatic feature selection: During the construction process of each tree, the random forest only randomly selects some features for node splitting, thereby reducing the interference of irrelevant features in high-dimensional data and improving model efficiency.

- No need for dimensionality reduction preprocessing: Unlike many traditional algorithms, random forests can directly process high-dimensional feature spaces (such as gene expression data or text features) without relying on dimensionality reduction techniques such as PCA.

- Processing Sparse Data: For sparse features (such as categorical variables encoded by one hot encoding), random forests effectively screen important features through indicators such as information gain or Gini impurity, avoiding dimensional disasters.

- Supporting mixed data types: Random forests can simultaneously handle numerical, categorical, and even missing data without the need for complex feature engineering or standardization steps.

Feature Importance Evaluation

01

Based on impurity reduction

Random forest quantifies the importance of features by calculating the reduction in Gini impurity or information entropy of each feature during node splitting, helping users understand the key driving factors of data.

02

Permutation importance analysis

By randomly shuffling the values of a certain feature and observing the degree of performance degradation of the model, it provides another reliable method for evaluating feature importance, especially suitable for data with nonlinear relationships.

03

Visual interpretability

With the help of tools such as Scikit learn's' feature_iimportances_ 'attribute, users can intuitively compare the contribution of different features, assisting feature filtering or business decision-making.

04 Applications of Random Forest



Classification Problems

- High dimensional data processing: Random forests can efficiently process high-dimensional feature data and reduce the risk of overfitting through the voting mechanism of multiple decision trees. They are suitable for fields such as gene expression analysis and image recognition. For example, in bioinformatics, it can be used to distinguish cancer subtypes or predict disease risk.

- Unbalanced dataset processing: By adjusting category weights or using undersampling/oversampling techniques, random forests can effectively solve the problem of uneven category distribution, such as the identification of rare fraudulent transactions in financial fraud detection.

- Multi class classification: supports multi class output without complex adjustments, suitable for text classification in natural language processing (such as sentiment analysis) or handwritten digit recognition (MNIST dataset).

Regression Tasks

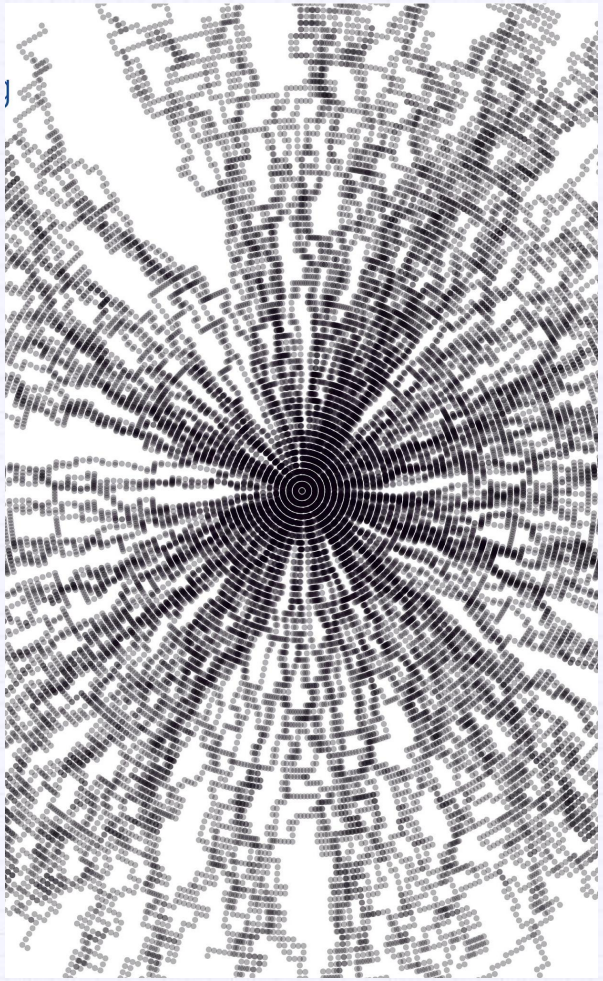
- Nonlinear relationship modeling: Random forests can capture complex nonlinear relationships between features and target variables, making them suitable for scenarios such as housing price prediction and sales trend analysis, and superior to traditional linear regression models.

- Missing value robustness: Through bootstrap and random feature selection, it has strong tolerance for missing data and is suitable for handling incomplete datasets in the real world, such as medical records.

- Variable importance assessment: It can output feature importance scores to assist in identifying key influencing factors, such as analyzing the impact of pollutants on ecological indicators in environmental science.

- Time series prediction: Built with lagged features, it can be used for short-term flow forecasting or energy demand estimation, but attention should be paid to the limitations when dealing with sequence dependencies.

Anomaly Detection



Unsupervised anomaly detection

By calculating the deviation between the sample and the decision tree path length (such as the Isolation Forest variant), identifying outliers in the data, suitable for network intrusion detection or industrial equipment fault warning.

High dimensional anomaly localization

In credit card transactions or sensor data, it can simultaneously process a large number of features and locate the source of anomalies, such as detecting abnormal consumption patterns or abnormal device readings.

Semi supervised learning adaptation

supports training with a small number of annotated abnormal samples to improve detection accuracy, suitable for rare lesion recognition in medical imaging or manufacturing defect detection.

Random Forest Formula

Classification Formula

For classification, the output is the **majority vote** (mode) of predictions from all trees:

$$\hat{y} = mode\{h_1(x), h_2(x), \ldots, h_B(x)\}$$

$\hat{y}$: Predicted class.

$h_b(x)$: Prediction of the b-th tree for input x.

B: Total number of trees (e.g., ntree = 100).

Regression Formula

For regression, the output is the **average** of all tree predictions:

$$\hat{y} = \frac{1}{B}\sum_{b=1}^{B} T_b(x; \Theta_b)$$

$T_b(x; \Theta_b)$: Prediction of the b-th tree with parameters Θ_b (e.g., random feature subset).

Key Parameters

ntree (or n_estimators): Number of trees. Default: 500 for classification, 100 for regression.

mtry (or max_features): Features considered per split. Default:

Classification: $\sqrt{p}$ (where p = total features).

Regression: $p/3$.

OOB Error: Out-of-bag error, an unbiased model performance estimate.

Example 1: Classification (Iris Dataset)

Example 1: Classification (Iris Dataset)

Task: Predict iris species (3 classes) using 4 features: sepal length/width, petal length/width.

Model Setup (from):

Data: 102 training samples (36 setosa, 31 versicolor, 35 virginica).

Parameters: ntree = 100, mtry = 2.

Output:

OOB error rate: **1.92%**.

Confusion matrix:

Actual \ Predicted	**setosa**	**versicolor**	**virginica**	**Error Rate**
setosa	36	0	0	0.00%
versicolor	0	31	1	3.125%
virginica	0	1	35	2.778%

Example 1: Classification (Iris Dataset)

Confusion matrix:

Actual \ Predicted	setosa	versicolor	virginica	Error Rate
setosa	36	0	0	0.00%
versicolor	0	31	1	3.125%
virginica	0	1	35	2.778%

Prediction Walkthrough:

For a new sample: [sepal_length=5.1, sepal_width=3.5, petal_length=1.4, petal_width=0.2]:

1.Tree 1 predicts: **setosa**.

2.Tree 2 predicts: **versicolor**.

3....

4.Tree 100 predicts: **setosa**.

Aggregation: 80 trees vote **setosa**, 15 vote **versicolor**, 5 vote **virginica**.

→ Final prediction: **setosa** (majority vote).

> **Why it works**: Feature randomness ensures diversity; majority voting reduces individual tree errors.

Example 2: Regression (Species Distribution Prediction)

Task: Predict species presence probability (0–1) using 7 environmental variables (e.g., temperature, humidity).

Model Setup (from):

Parameters: ntree = 500, mtry = 2, regression mode.

Performance Metrics:

Mean Squared Error (MSE): **0.07086081**.

% Variance Explained: **67.42%** (pseudo-R^2).

RMSE: $\sqrt{0.07086081} \approx 0.266$.

Prediction Walkthrough:

For a new sample: [var1=0.5, var2=0.3, ..., var7=0.6]:

1.Tree 1 predicts: **0.72**.

2.Tree 2 predicts: **0.68**.

3....

4.Tree 500 predicts: **0.71**.

Aggregation:

$$\hat{y} = \frac{0.72 + 0.68 + \cdots + 0.71}{500} \approx 0.70$$

→ Final prediction: **0.70** (species presence probability).

> **Why it works**: Averaging tree outputs smooths noise and improves stability. High R^2 (67.42%) indicates strong predictive power.

Key Insights from Evidence

1.**Error Rates Vary by Parameters**:

- Higher ntree (e.g., 500) often lowers error (e.g., 4% in vs. 5.41% in for Iris).
- Poor parameter tuning can raise errors (e.g., 39.13% in).

2.**Feature Importance**:

- In Iris data, petal length is often the most significant predictor .

3.**OOB Error Reliability**:

- OOB estimates (e.g., 1.92% in) closely match test errors, eliminating the need for a separate validation set.

Classification: Use majority vote. Example: Iris prediction with 1.92% OOB error.

Regression: Use averaging. Example: Species distribution with 67.42% variance explained.

Critical Step: Tune ntree and mtry to balance accuracy/computational cost. Implementations (e.g., R/Python) handle formulas internally, but understanding aggregation is key to interpreting results.

Sources: (error comparison), (feature importance), (OOB validity).

05 Comparison with Other Algorithms



Random Forest vs. Decision Trees



Advantages of ensemble learning

Random forest significantly reduces the risk of overfitting for a single decision tree by constructing multiple decision trees and synthesizing voting results, improving the model's generalization ability and stability.



Random feature selection

Unlike single decision trees that use all features, random forests only randomly select some features at the split nodes of each tree, further enhancing the diversity and noise resistance of the model.



Computational efficiency

Although random forests require training multiple trees, due to the parallelization of the training process for each tree and the fact that a single tree is usually shallow, the overall computational efficiency is still better than that of deeply optimized single decision trees.

Random Forest vs. Gradient Boosting

- Differences in training methods: Random forest uses Bagging (self aggregation) method to independently train each tree, while gradient boosting (such as XGBoost) trains iteratively in series, with the latter tree correcting the residuals of the previous tree.

- Overfitting control: Random forests improve performance by increasing the number of trees without overfitting; Gradient boosting requires careful control of learning rate and tree depth, otherwise it is easy to become overly sensitive to noisy data.

- Prediction speed: The prediction process of random forests can be fully parallelized, making it suitable for real-time applications; Gradient boosting has a relatively high prediction delay due to serial dependency.

- Parameter sensitivity: Gradient boosting requires more precise adjustment of hyperparameters (such as learning rate and subsampling ratio), while random forests have stronger robustness to parameter changes, and default parameters often achieve better results.

Random Forest vs. Support Vector Machines

Data adaptability

Random forests naturally support high-dimensional sparse data (such as text features) without feature scaling, while SVM is sensitive to feature scale and kernel function selection directly affects performance.

Nonlinear processing

SVM implicitly maps high-dimensional space through kernel techniques to handle nonlinear problems, and the computational complexity increases with the sample size; **Random forests explicitly model nonlinearity through feature randomness and tree structure**, making them **suitable for large-scale data**.

Explanatory

Random forests provide interpretable tools such as feature importance ranking and partial dependency graphs, while SVM's decision boundaries (especially when using complex kernel functions) are often difficult to understand intuitively.

06 Practical Implementation Tips



Hyperparameter Tuning

- N_estimators (number of decision trees): Increasing the number of trees usually improves model performance, but it also increases computational costs. It is recommended to select a balance point through cross validation, with a common range of 100-500.

- Max_depth (maximum depth of tree): Limiting the depth of the tree can prevent overfitting. You can try 3-10 layers through grid search, or set it to None to allow the tree to grow completely (subject to other parameter control).

- Min_Samples_split: Set the minimum number of samples required for node splitting (default 2), with higher values (such as 5-10) simplifying the model and reducing noise impact.

- Max_features: Controls the number of features considered when splitting each tree, commonly using 'sqrt (n_features)' or 'log2'. Reducing this value can enhance diversity, but may sacrifice accuracy.

Handling Missing Values

Built in missing value handling

Random Forest supports direct handling of missing values (such as SimpleImperter in sklearn), by calculating the direction of missing values during splitting or assigning them to branches with higher frequency of occurrence.

Filling strategy

Numerical missing data can be filled with median/mean, while categorical missing data can be filled with mode or "unknown" labels. Avoid deleting missing samples to prevent data bias.

Surrogate Splits

Partial implementations (such as' randomForest 'in R) allow surrogate splits to replace the main splitting rules and preserve the information utilization of missing samples.

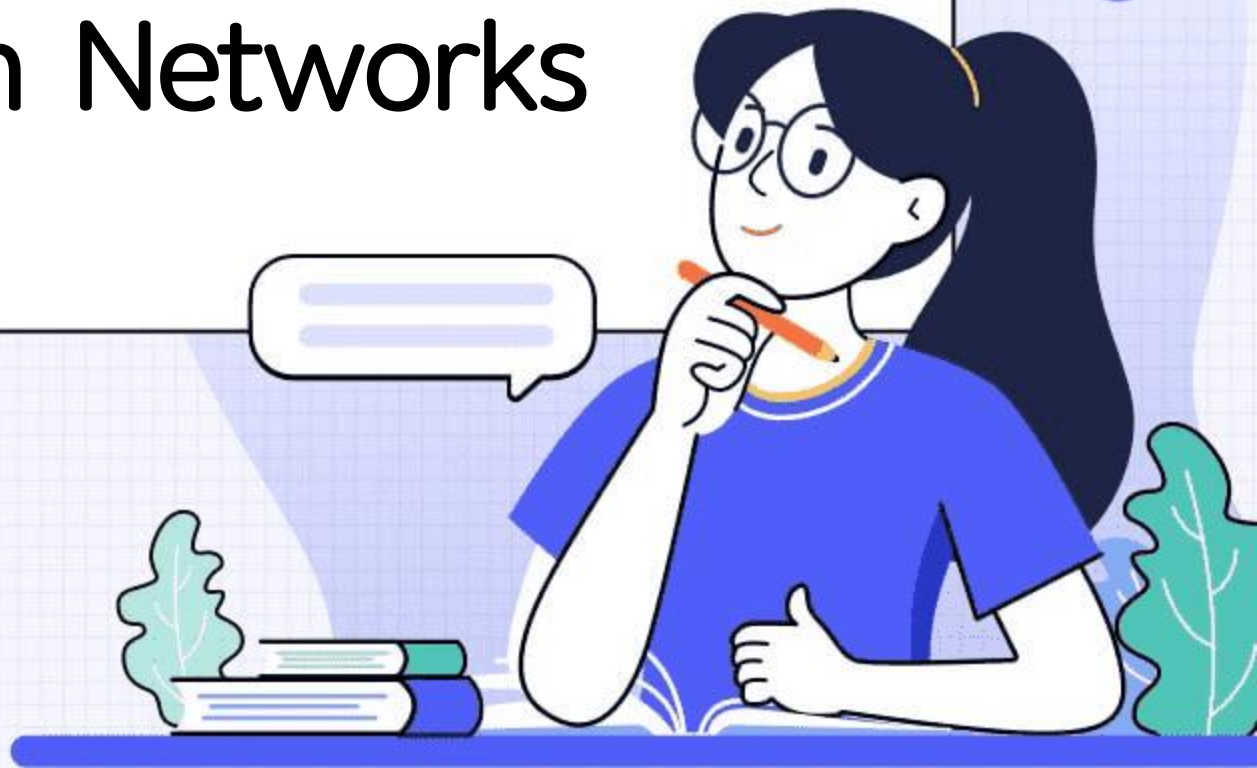
Performance Optimization Techniques

- Parallel training: Utilizing the 'n_jobs' parameter to allocate multi-core CPUs (such as using all cores with' n_jobs=-1 ') significantly accelerates large-scale dataset training.
- Feature importance screening: By using 'feature*iimportances*' to remove low contribution features, it reduces computational complexity and may improve generalization ability.
- Incremental learning (Warm Start): Setting 'warm_start=True' can increase the number of trees without resetting the model, making it suitable for dynamically adjusting resources.
- Early Stopping Mechanism: Monitor the performance of the validation set and terminate training prematurely when new trees no longer improve accuracy (requires custom implementation or integration with the 'scikit learn' extension).



01

Introduction to Bayesian Networks



Definition and Origin

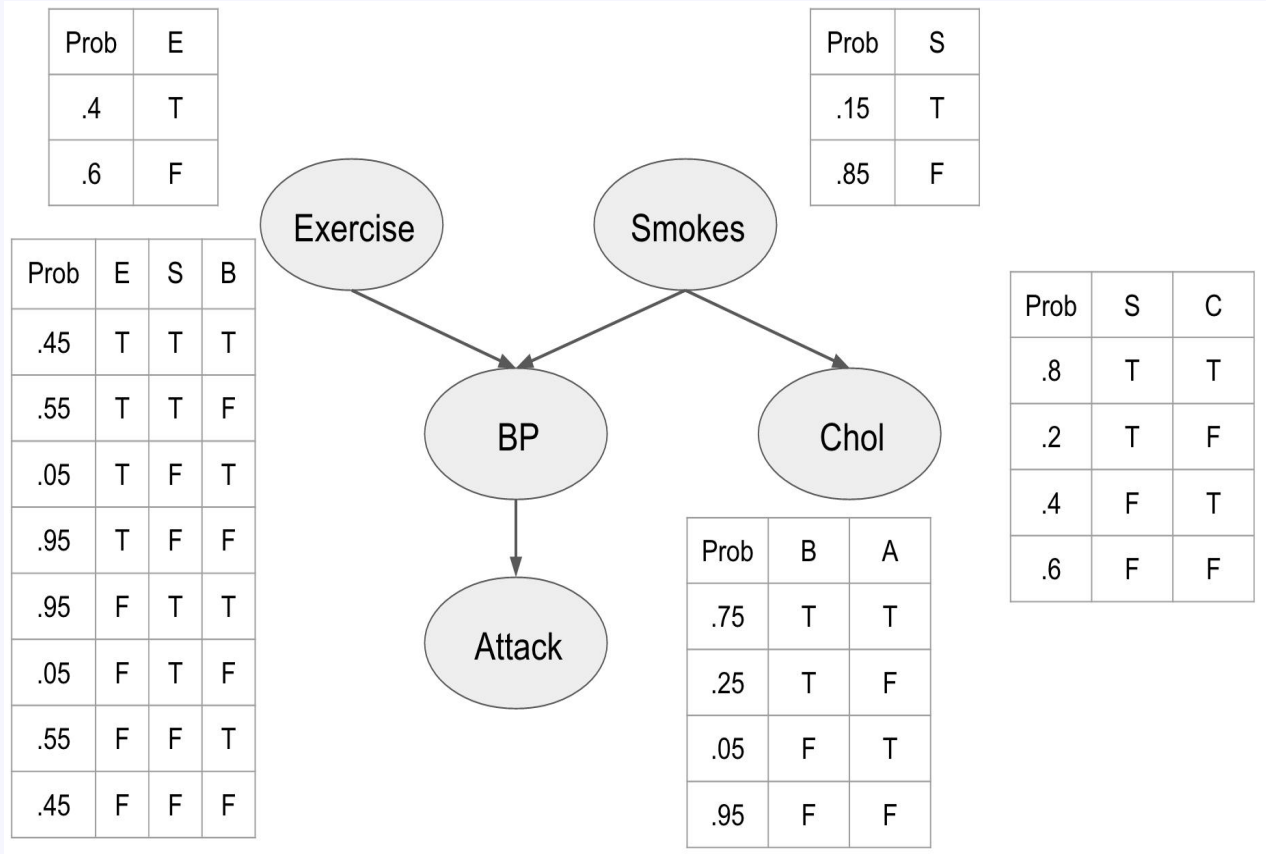
- **Probability graph model: Bayesian network is a statistical model** that uses **directed acyclic graph (DAG) to represent the probability dependency relationships** between variables. **Nodes represent random variables, edges represent conditional dependency relationships**, and efficient computation is achieved through joint probability decomposition.

- **Historical development:** Originating from the causal reasoning theory proposed by Judea Pearl in the 1980s, Finn V. Jensen's 1996 monograph "An Introduction to Bayesian Networks" systematized its mathematical foundation, integrating graph theory and Bayesian statistical theory.

- **Core idea:** Based on **Bayes' theorem**, combining prior knowledge with observed data, achieving uncertainty inference through posterior probability updates, especially adept at **handling incomplete data scenarios**.

Bayesian Network

- This network shows how lifestyle choices (exercise and smoking) influence risk factors (BP and cholesterol), which in turn affect the probability of a heart attack.
  - You can use it to:
  - Predict heart attack risk based on lifestyle
  - Diagnose likely causes given a heart attack
  - Evaluate impact of lifestyle changes



Key Characteristics

- **Conditional independence assumption:** By using the d-separation criterion to determine variable independence, the computational complexity is significantly reduced. For example, the **Naive Bayes classifier assumes complete independence between features**.

- **Prior knowledge integration:** allows expert experience to be embedded in the model in the form of prior probabilities, enhancing generalization ability on small sample data, such as symptom disease relationship modeling in medical diagnosis.

- **Dynamic reasoning ability:** supports evidence propagation, updates node probabilities in real-time, and is suitable for dynamic systems such as fault diagnosis or real-time decision-making systems.

- **Interpretability advantage:** The network structure intuitively displays causal relationships between variables, making it easier to verify logical rationality compared to black box models, such as variable impact analysis in financial risk assessment.

Applications in Machine Learning

1 Classification task

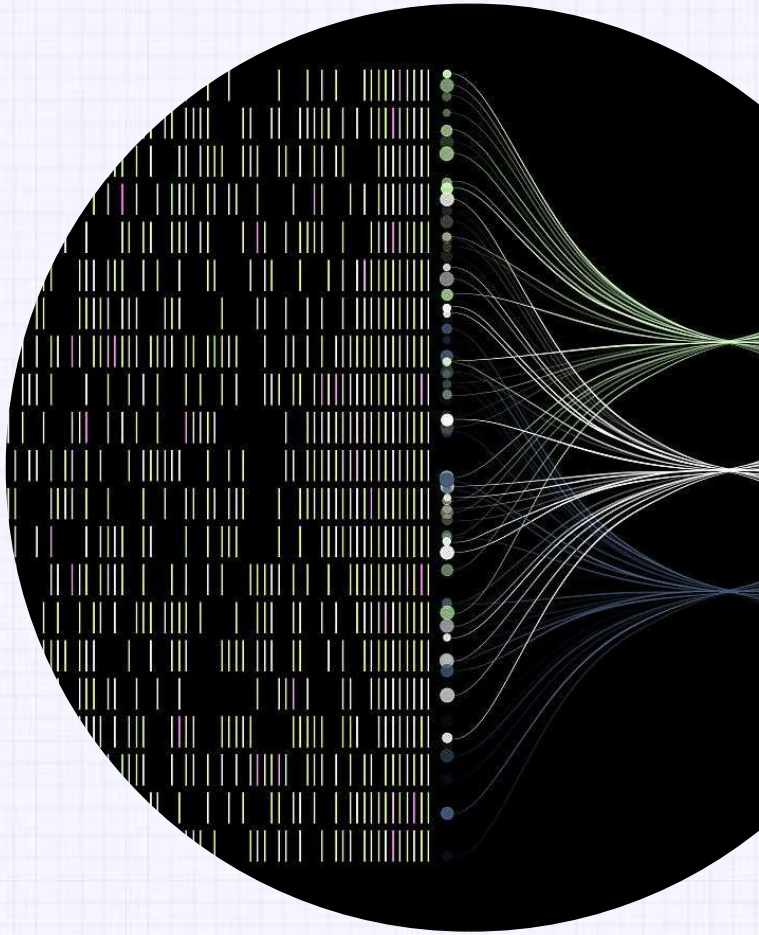
Naive Bayes classifier is widely used for text classification (such as spam filtering), achieving efficient prediction through word frequency conditional probability calculation. Although simple, it has strong baseline performance.

2 Anomaly detection

Construct variable dependent networks (such as sensor data streams), identify behaviors that deviate from normal patterns through probability anomaly scores, and apply them to industrial equipment fault warning.

3 Recommendation system

Build a probability graph model by combining user behavior data and item attributes to optimize the cold start problem of collaborative filtering, such as Netflix's hybrid recommendation algorithm.



02

Bayesian Theorem Fundamentals



Probability Basics

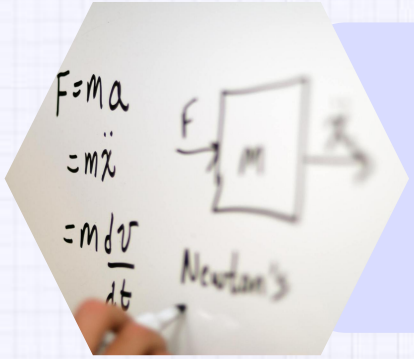
- **Sample space and events:** The foundation of probability theory is the sample space (the set of all possible outcomes) and events (subsets of the sample space), which describe the regularity of random phenomena by defining probability measures of events.

- **Conditional probability:** describes the probability of another event occurring under known conditions, with the formula $P(A \mid B)=P(A \cap B)/P(B)$, which is the core component of **Bayes' theorem**.

- **Independence and correlation:** The independence of two events means that the occurrence of one event does not affect the probability of the other, while correlation measures the degree of statistical correlation between events.

- **Random variables and distribution:** Random variables map the sample space to real numbers, and their probability distributions (such as binomial distribution and normal distribution) describe the probability patterns of random variable values.

Bayes' Rule Explained

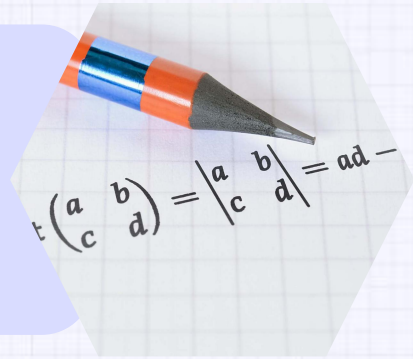


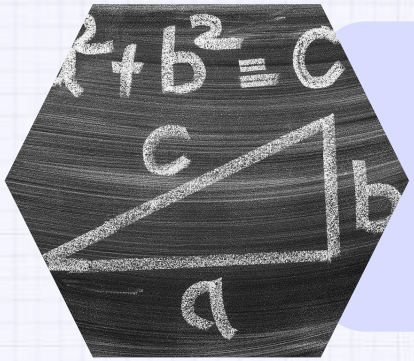
Theorem formula

The Bayesian formula is P (A | B)=[P (B | A) P (A)]/P (B), where P (A) is the prior probability, P (B | A) is the likelihood, P (A | B) is the posterior probability, and P (B) is the marginal probability.

Inverse probability problem

Bayes' theorem updates the probability estimation of events through prior knowledge and new evidence (data), solving the inverse probability problem that is difficult to handle in traditional probability theory.





Application scenarios

Widely used in medical diagnosis (such as disease detection), spam filtering, machine learning model parameter estimation, and other fields, reflecting the idea of "dynamically updating cognition".

Prior vs. Posterior Probability

- **Prior probability:** The initial probability assumption of a parameter based on historical knowledge or subjective judgment before observing data, such as assuming a positive probability of P=0.5 when coins are uniformly distributed.

- **Posterior probability:** The modified probability obtained by combining the prior probability with the observed data (likelihood), reflecting the adjustment result of the data on the initial belief.

- **Conjugate prior:** To simplify the calculation, select a prior distribution that matches the form of the likelihood function (such as the Beta distribution as the conjugate prior of the binomial distribution), so that the posterior distribution belongs to the same family as the prior.

- **The influence of information quantity:** The strength of prior (such as high variance indicating weak prior) will significantly affect the posterior results, and the larger the data quantity, the less the influence of prior on the posterior.

Bayes' Rule Equation

Bayes' Rule calculates the **posterior probability** $P(A|B)$—the updated likelihood of event A occurring after observing evidence B. The core formula is:

$$\boxed{P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}}$$

Key Components:

- $P(A|B)$: Posterior probability (probability of A given B).
- $P(B|A)$: Likelihood (probability of B given A).
- $P(A)$: Prior probability (initial probability of A before evidence).
- $P(B)$: Marginal probability (total probability of B), calculated as:

$$P(B) = \sum_i P(B|A_i) \cdot P(A_i)$$

where A_i partitions the sample space (e.g., A and "not A").

Bayes' Rule Equation - Extended Form (for Multiple Hypotheses)

If the sample space is divided into mutually exclusive events $A_1, A_2, \ldots, A_n$:

$$P(A_i|B) = \frac{P(B|A_i) \cdot P(A_i)}{\sum_{j=1}^{n} P(B|A_j) \cdot P(A_j)}$$

Why This Matters:

- Bayes' Rule reverses conditional probabilities (e.g., from "probability of a positive test given disease" to "probability of disease given a positive test").
- It quantifies how prior beliefs ($P(A)$) should rationally update with new data (B).
- Sources: The basic formula is consistent across and , while the partitioned form is emphasized in and .

Bayes' Rule Example: Medical Diagnosis

Scenario:

- A tumor malignancy test has high accuracy, but cancer is rare in the population.
- **Events**:
  - H: Tumor is malignant (the hypothesis).
  - E: Test result is positive (the evidence).
- **Given Probabilities**:
  - Prior probability of malignancy: $P(H) = 0.01$ (1% of people have malignant tumors).
  - Prior probability of no malignancy: $P(\neg H) = 1 - P(H) = 0.99$.
  - Test sensitivity (true positive rate): $P(E|H) = 0.90$ (90% chance test is positive if malignant).
  - Test false positive rate: $P(E|\neg H) = 0.10$ (10% chance test is positive if benign).

Goal:

Find $P(H|E)$: Probability that the tumor is malignant given a positive test result.

Goal:

Find $P(H|E)$: Probability that the tumor is malignant given a positive test result.

Step-by-Step Calculation:

1. **Compute the marginal probability** $P(E)$ (total probability of a positive test):

$$P(E) = P(E|H) \cdot P(H) + P(E|\neg H) \cdot P(\neg H)$$

Substitute the values:

$$P(E) = (0.90 \times 0.01) + (0.10 \times 0.99) = 0.009 + 0.099 = 0.108$$

2. **Apply Bayes' Rule** to find $P(H|E)$:

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)} = \frac{0.90 \times 0.01}{0.108} = \frac{0.009}{0.108} \approx 0.0833$$

Result: $P(H|E) \approx 0.0833$ or **8.33%**.

Bayes' Rule Example: Medical Diagnosis

Goal:

Find $P(H|E)$: Probability that the tumor is malignant given a positive test result.

Step-by-Step Calculation:

1. **Compute the marginal probability** $P(E)$ (total probability of a positive test):

$$P(E) = P(E|H) \cdot P(H) + P(E|\neg H) \cdot P(\neg H)$$

Substitute the values:

$$P(E) = (0.90 \times 0.01) + (0.10 \times 0.99) = 0.009 + 0.099 = 0.108$$

2. **Apply Bayes' Rule** to find $P(H|E)$:

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)} = \frac{0.90 \times 0.01}{0.108} = \frac{0.009}{0.108} \approx 0.0833$$

Result: $P(H|E) \approx 0.0833$ or **8.33%**.

Interpretation:

- Despite a positive test (E) and high test sensitivity (90%), the probability of malignancy is only **8.33%**.
- **Why?** The low prior probability of cancer ($P(H) = 1\%$) and significant false positives ($P(E|\neg H) = 10\%$) dominate the result. This highlights:
  - Blindly trusting "90% accurate" tests can be misleading without considering base rates (prior probabilities).
  - Bayes' Rule forces integration of prior knowledge (e.g., disease prevalence) with new evidence.

Real-World Insight:

This example mirrors real medical dilemmas (e.g., HIV or cancer screening). If doctors ignore priors, they may recommend unnecessary treatments. Sources like and emphasize this in clinical decision-making contexts.

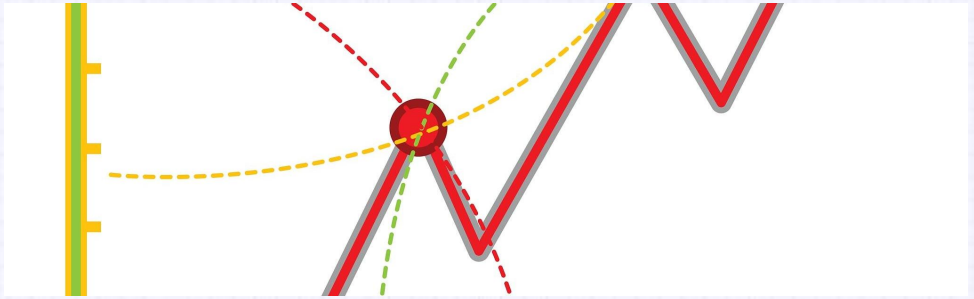
03 Naive Bayes Classifier



Model Assumptions

The assumption of feature independence

The core assumption of Naive Bayes is that all features are independent of each other under given category conditions, that is, the appearance of one feature does not affect the probability of the appearance of other features. This assumption simplifies joint probability calculations, but in reality, there may be dependencies between features.







Model Assumptions

- **Category prior probability:** The model needs to estimate the prior probability of each category in advance (such as the initial ratio of spam to non spam), usually calculated by the frequency of categories in the training data.

- **Gaussian distribution assumption (continuous features):** For continuous features, Naive Bayes often assumes that they follow a Gaussian distribution and estimates the conditional probability density by calculating the mean and variance.

- **Polynomial distribution assumption (discrete features):** In scenarios such as text classification, features are usually word frequency counts. In this case, polynomial distribution modeling is used to calculate likelihood probability through word frequency statistics.

Conditional Independence

01

Simplify computational complexity

The conditional independence assumption decomposes high-dimensional joint probabilities into the product of edge probabilities of each feature (such as $P(X_1, X_2 \mid C) = P(X_1 \mid C)\, P(X_2 \mid C)$), avoiding the difficulty of directly calculating exponential level combinations.

02

Data sparsity problem

When certain feature combinations do not appear in the training set, the independence assumption allows for filling missing values through probability estimates of individual features, but may introduce bias.

03

Compromise in practical applications

Despite the overly strict assumption of independence (such as the possible correlation between "winning" and "clicking" in spam emails), Naive Bayes is still widely used due to its efficiency and small sample size advantage.

Naive Bayes Classifier: Core Equation

1.**Bayes' Theorem Foundation**:

$$P(c/x) = \frac{P(x/c) \cdot P(c)}{P(x)}$$

$P(c/x)$: Posterior probability of class c given features x.

$P(c)$: Prior probability of class c.

$P(x/c)$: Likelihood of features x given class c.

$P(x)$: Evidence (marginal probability of features; **ignored in classification** as it is constant for all classes).

Naive Bayes Classifier: Core Equation

1. **"Naive" Assumption (Conditional Independence)**:

If $x = (x_1, x_2, \ldots, x_n)$ consists of n features:

$$P(x/c) = \prod_{i=1}^{n} P(x_i/c)$$

Classifier Decision Rule:

$$\hat{c} = \text{argmax}_c \left[P(c) \cdot \prod_{i=1}^{n} P(x_i/c) \right]$$

Naive Bayes Classifier: Core Equation

1.**Probability Estimation Methods**:

Basic Estimate: $P(x_i/c) = \frac{N_{ic}}{N_c}$

(N_{ic}: Count of samples with feature x_i in class c; N_c: Total samples in class c).

Laplace Smoothing (handles zero probabilities):

$$P(x_i/c) = \frac{N_{ic} + 1}{N_c + k}$$

(k: Number of possible values for feature x_i).

m**-Estimate** (generalized smoothing):

$$P(x_i/c) = \frac{N_{ic} + m \cdot p}{N_c + m}$$

(p: Prior estimate of $P(x_i/c)$, often $p = \frac{1}{k}$; m: Smoothing parameter).

Example 1: Weather-Based Activity Prediction (Discrete Features)

Task: Predict if outdoor activity is suitable ("Play=Yes" or "Play=No") for conditions:

X = (Outlook=Sunny, Temperature=Cool, Humidity=High, Windy=False).

Training Data (14 samples):

Class Prior Probabilities:

$P(Play = Yes) = \frac{9}{14}$, $P(Play = No) = \frac{5}{14}$.

Feature Likelihoods:

Feature	$P(\cdot \mid Yes)$	$P(\cdot \mid No)$
Outlook=Sunny	$\frac{2}{9}$	$\frac{3}{5}$
Temperature=Cool	$\frac{3}{9}$	$\frac{1}{5}$
Humidity=High	$\frac{3}{9}$	$\frac{4}{5}$
Windy=False	$\frac{6}{9}$	$\frac{2}{5}$

Example 1: Weather-Based Activity Prediction (Discrete Features)

Feature	$P(\cdot \mid Yes)$	$P(\cdot \mid No)$
Outlook=Sunny	$\frac{2}{9}$	$\frac{3}{5}$
Temperature=Cool	$\frac{3}{9}$	$\frac{1}{5}$
Humidity=High	$\frac{3}{9}$	$\frac{4}{5}$
Windy=False	$\frac{6}{9}$	$\frac{2}{5}$

Posterior Probability Calculation:

For **Play=Yes**:

$$P(Yes|X) \propto P(Yes) \cdot \prod P(x_i|Yes) = \frac{9}{14} \times \frac{2}{9} \times \frac{3}{9} \times \frac{3}{9} \times \frac{6}{9} = 0.01$$

For **Play=No**:

$$P(No|X) \propto P(No) \cdot \prod P(x_i|No) = \frac{5}{14} \times \frac{3}{5} \times \frac{1}{5} \times \frac{4}{5} \times \frac{2}{5} = 0.013$$

Result: Since $P(No|X) > P(Yes|X)$, classify as **"Play=No"** (not suitable).

Example 2: Loan Default Prediction (Mixed Discrete & Continuous Features)

Task: Predict if a borrower will default ("Default=Yes" or "Default=No") for:

X = (HomeOwner=No, MaritalStatus=Married, AnnualIncome=$120K).

Training Data (10 samples):

Class Prior Probabilities:

$P(Default = Yes) = \frac{3}{10} = 0.3,\ P(Default = No) = \frac{7}{10} = 0.7.$

Discrete Feature Likelihoods:

Feature	$P(\cdot \mid Yes)$	$P(\cdot \mid No)$
HomeOwner=No	$\frac{1}{1}$	$\frac{4}{7}$
MaritalStatus=Married	$\frac{0}{1}$	$\frac{4}{7}$

Continuous Feature (AnnualIncome):

Assume **Gaussian distribution** (calculate mean μ and variance σ^2 per class):

Class	μ	σ^2
Yes	90	25
No	110	2975

Example 2: Loan Default Prediction (Mixed Discrete & Continuous Features)

Discrete Feature Likelihoods:

Feature	$P(\cdot \mid Yes)$	$P(\cdot \mid No)$
HomeOwner=No	$\frac{1}{1}$	$\frac{4}{7}$
MaritalStatus=Married	$\frac{0}{1}$	$\frac{4}{7}$

Continuous Feature (AnnualIncome):

Assume **Gaussian distribution** (calculate mean μ and variance σ^2 per class):

Class	μ	σ^2
Yes	90	25
No	110	2975

Likelihood for Income=120 in class **No**:

$$P(120|No) = \frac{1}{\sqrt{2\pi \cdot 2975}} \exp\left(-\frac{(120-110)^2}{2 \cdot 2975}\right) \approx 0.0072$$

Posterior Calculation (with Laplace Smoothing for $P(Married|Yes)$):

For **Default=Yes**:

$$P(Yes|X) \propto 0.3 \times \frac{1}{1} \times red\frac{0+1}{1+3} \times P(120|Yes) = 0.3 \times 1 \times 0.25 \times (smallvalue) \approx 0$$

(Note: $P(Married|Yes)$ smoothed to $\frac{0+1}{1+3} = 0.25$).

For **Default=No**:

$$P(No|X) \propto 0.7 \times \frac{4}{7} \times \frac{4}{7} \times 0.0072 = 0.0024$$

Result: $P(No|X)$ is higher → classify as **"Default=No"** (will not default).

Spam Filtering Example

Feature engineering

Convert email content into a Bag of Words model, where each word appears as a binary feature (such as "free" or "discount"), and calculate the contribution of high-frequency words to classification.

Probability calculation

Based on the historical email database, calculate the conditional probability of each word in spam and normal emails (such as P ("free" | spam)=0.8, P ("free" | normal)=0.1).

Posteriori decision

After extracting features from new emails, calculate the posterior probability based on Bayesian formula (such as P (junk | "free, click")). If it exceeds the threshold, it is judged as spam.

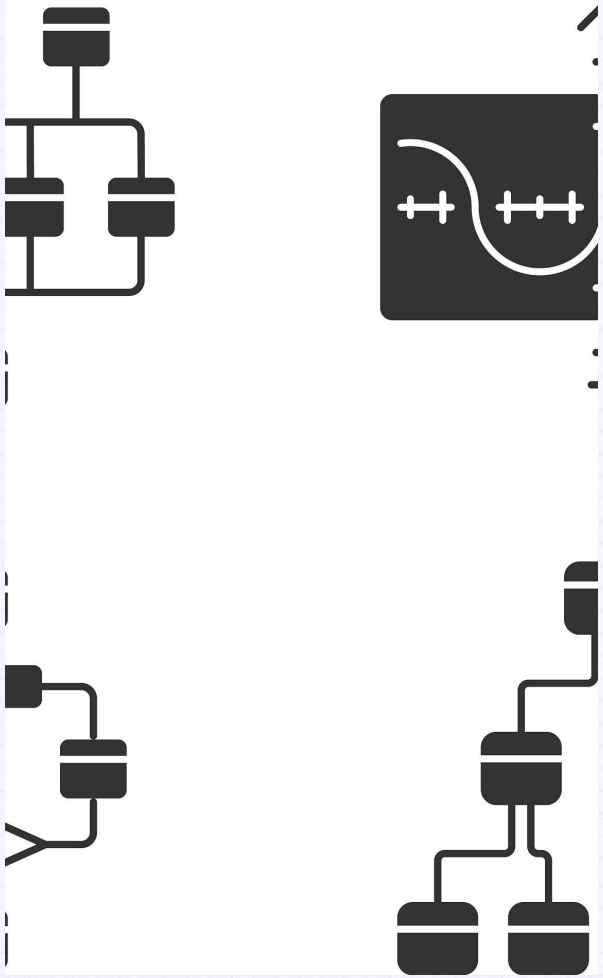
Laplace smoothing

To avoid the probability of zero caused by unregistered words (such as newly appearing advertising words), smoothing techniques (such as adding 1 smoothing) are introduced to ensure the robustness of the model.

04 Bayesian Network Inference



Exact Inference Methods



Variable Elimination

A systematic approach to marginalize out variables by summing over their possible values, reducing the computational complexity of inference in Bayesian networks. It involves rearranging the order of summations and products to optimize efficiency.

Junction Tree Algorithm

Constructs a junction tree from the Bayesian network to enable efficient exact inference. This method involves moralizing the graph, triangulating it, and then performing message passing on the resulting tree structure.

Sum-Product Algorithm

A message-passing algorithm used for exact inference on tree-structured graphs. It efficiently computes marginal distributions by passing messages between nodes and combining them to obtain the desired probabilities.

Approximate Inference Techniques

- **Markov Chain Monte Carlo (MCMC):** A family of algorithms that approximate the posterior distribution by sampling from it. Techniques like Gibbs sampling and Metropolis-Hastings are commonly used to explore the state space and generate samples.

- **Variational Inference:** Approximates the posterior distribution by optimizing a simpler, tractable distribution to minimize the Kullback-Leibler divergence from the true posterior. This method is often faster than MCMC but may sacrifice accuracy.

- **Loopy Belief Propagation:** An extension of the sum-product algorithm to graphs with cycles. Although not guaranteed to converge, it often provides reasonable approximations in practice by iteratively passing messages until stability is reached.

- **Importance Sampling:** Estimates the posterior distribution by weighting samples drawn from a proposal distribution. This method is particularly useful when direct sampling from the posterior is infeasible, but it can suffer from high variance.

Markov Chain Monte Carlo (MCMC)

MCMC methods are primarily used for **calculating numerical approximations of multi-dimensional integrals**, for example in Bayesian statistics, computational physics, computational biology, and computational linguistics.

Key Concepts:

1.Markov Chain: A sequence of random variables where the future state depends only on the current state, not on the sequence of events that preceded it (memoryless property).
2.Stationary Distribution: The limiting distribution of the Markov chain, which is the distribution we want to sample from.
3.Detailed Balance: A condition that ensures the Markov chain converges to the desired stationary distribution.

Common MCMC Algorithms:

1.Metropolis-Hastings Algorithm: A general algorithm that uses a proposal distribution to generate candidate states and an acceptance/rejection step to ensure convergence to the target distribution.
2.Gibbs Sampling: A special case of Metropolis-Hastings where the proposal distribution is the conditional distribution of each variable given the current values of all other variables.
3.Hamiltonian Monte Carlo (HMC): Uses Hamiltonian dynamics to propose new states, reducing random walk behavior and improving efficiency.
4.No-U-Turn Sampler (NUTS): An extension of HMC that automatically tunes the number of steps to avoid retracing the path.

Real-world Use Cases

Medical Diagnosis

Bayesian networks are used to model the **probabilistic relationships between symptoms and diseases**, enabling doctors to make informed diagnoses based on observed evidence and prior knowledge.

Spam Filtering

Bayesian methods are employed to classify emails as spam or not spam by **learning the probability distributions of words in both categories** and updating beliefs as new data arrives.

Autonomous Vehicles

Bayesian inference helps in **sensor fusion and decision-making by combining uncertain sensor data** with prior models to estimate the state of the environment and predict future events.

05 Learning Bayesian Networks



Parameter Learning



Maximum Likelihood Estimation (MLE)

Estimating the parameters of a Bayesian network **by maximizing the likelihood function of observed data**, suitable for complete datasets and assuming independent and identically distributed data. It is computationally efficient but sensitive to small samples.



Bayesian estimation

Introducing prior distributions (such as Dirichlet distribution) to model parameters, combined with posterior distributions for parameter estimation, can effectively handle data sparsity problems but has high computational complexity.



Expectation Maximization (EM) algorithm

Iteratively optimizes the parameters of the latent variable model, suitable for situations with hidden variables or missing data. It gradually converges to the local optimal solution by calculating the **expectation in E-step** and maximizing the **likelihood in M-step**.

Maximum Likelihood Estimation (MLE)

- A foundational method in statistics for estimating the parameters of a probability model.
- It works by finding the parameter values that maximize the likelihood.
- That is, the probability of observing the given data under the assumed model.

🔍 Core Idea

Given:

- A dataset $\mathcal{D} = \{x_1, x_2, \ldots, x_n\}$
- A parametric model $P(x \mid \theta)$

We want to find the parameter θ that makes the observed data **most probable**:

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} P(\mathcal{D} \mid \theta)$$

🧮 Likelihood Function

Assuming the data are **i.i.d.** (independent and identically distributed), the likelihood is:

$$L(\theta) = \prod_{i=1}^{n} P(x_i \mid \theta)$$

In practice, we often work with the **log-likelihood** for numerical stability:

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^{n} \log P(x_i \mid \theta)$$

Maximum Likelihood Estimation (MLE)

✅ **MLE Steps**

1. **Write the likelihood** $L(\theta)$
2. **Take the log** to get $\ell(\theta)$
3. **Differentiate** $\ell(\theta)$ with respect to θ
4. **Set derivative to zero** and solve for θ

📌 **Example: Bernoulli Distribution**

Suppose we have n coin flips: $x_i \in \{0, 1\}$, modeled as:

$$P(x_i \mid p) = p^{x_i}(1-p)^{1-x_i}$$

Log-likelihood:

$$\ell(p) = \sum_{i=1}^{n} [x_i \log p + (1 - x_i) \log(1-p)]$$

Take derivative and solve:

$$\frac{d\ell}{dp} = \sum_{i=1}^{n} \left(\frac{x_i}{p} - \frac{1-x_i}{1-p} \right) = 0 \Rightarrow \hat{p}_{\text{MLE}} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

Bayesian Estimation

- A statistical method that updates the probability for a hypothesis as more evidence or information becomes available.
- It's named after Thomas Bayes, an 18th-century mathematician and theologian.

- Bayesian Estimation provides a **powerful and flexible framework for statistical inference**, allowing you to incorporate prior knowledge and update beliefs systematically as new data becomes available.

- While it has computational and interpretational challenges, its benefits in uncertainty quantification and flexibility make it highly valuable across many fields.

Expectation Maximization (EM) algorithm

- A general-purpose technique for **maximum-likelihood (or maximum-a-posteriori) estimation** when the data are incomplete or partially observed.

- **When should you use EM?**
  - Natural latent-variable structure (missing data, component memberships, censored observations).
  - Complete-data likelihood is easy (exponential family is ideal).
  - Moderate dimensionality; very high-dimensional latent spaces may require variational or sampling approximations.

Structure Learning

- **Constraint based approach:** constructing network structures through statistical independence tests (such as chi square tests), represented by **PC algorithm and IC algorithm**, suitable for sparse networks but sensitive to high-dimensional data.

- **Scoring based method:** Define scoring functions (such as BIC, AIC) to evaluate network quality, and combine search strategies (greedy search, simulated annealing) to find the optimal structure, balancing fitting goodness and model complexity.

- **Hybrid method:** Combining constraints and scoring strategies, first using independence tests to narrow down the search space, and then refining the structure with a scoring function. The typical representative is the **MMHC algorithm**, which combines efficiency and accuracy.

- **Dynamic structural learning:** A **dynamic Bayesian network (DBN)** for time-series data that captures the temporal dependencies between variables through sliding windows or state space models.

Constraint based approach - PC and IC algorithms

- PC (Peter-Clark)
  - The IC algorithm is a foundational constraint-based algorithm developed by Pearl and Verma for learning causal networks.

- IC (Inductive Causation)
  - Named after Peter Spirtes and Clark Glymour, was developed as a more computationally efficient alternative to the IC algorithm.

Scoring based method: BIC, AIC

- AIC is prediction-oriented (best average out-of-sample likelihood).
  - estimate of the expected Kullback-Leibler divergence between the fitted model and the (unknown) true generating density, multiplied by 2 n.

- BIC is explanation-oriented (highest posterior probability if the true model is among the candidates).
  - −2 · log of the Bayes factor (under a unit-information prior) versus the null model, i.e. an approximation to −2 · log marginal likelihood.

Hybrid method - MMHC algorithm

- MMHC (Max-Min Hill-Climbing) is a hybrid structure-learning algorithm that marries a constraint-based phase with a score-based phase.

- Goal to build an undirected skeleton that contains every true edge and as few false edges as possible.

- MMHC is the prototypical "best of both worlds" solution:
  - Use constraints to throw away the obviously wrong edges, then use scoring to polish the remaining structure.

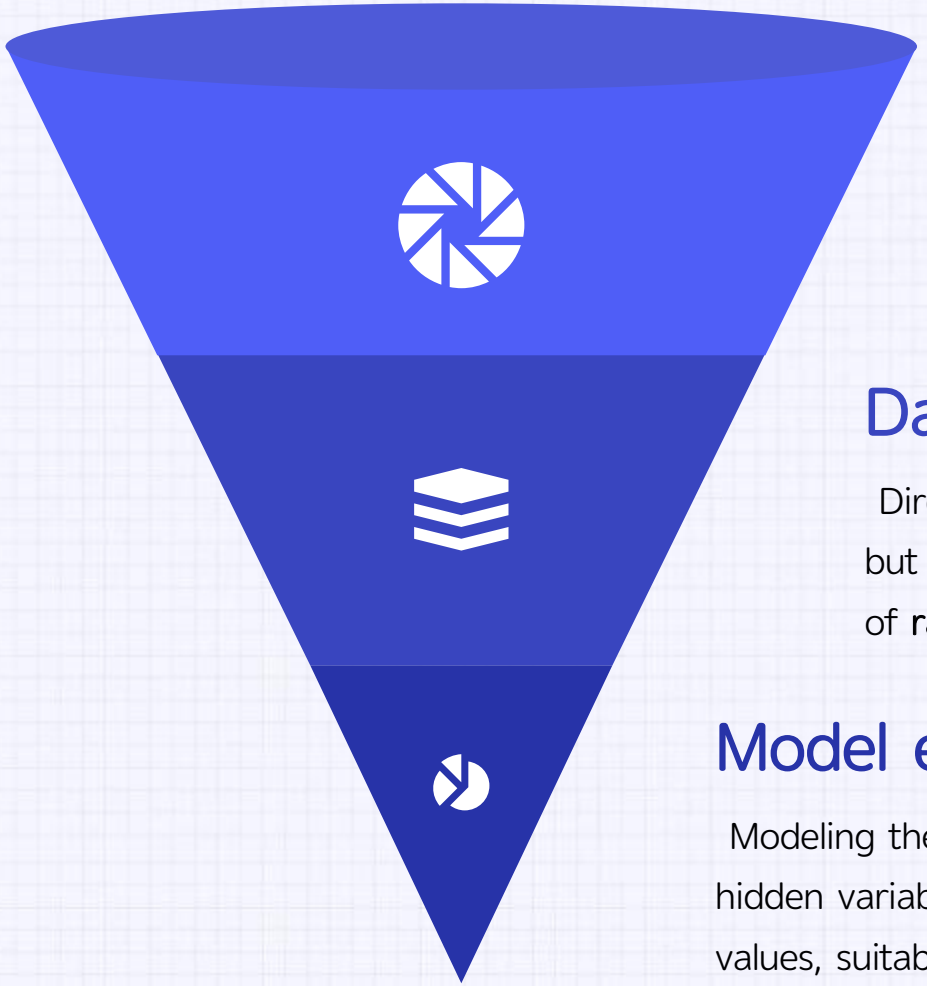
Dynamic Bayesian Networks (DBNs)

- An extension of Bayesian Networks (BNs) that model temporal or time-series data.

A DBN is defined by two main components:

- **Prior Network (Time Slice 0)**: A BN representing the initial distribution of variables at time $t = 0$.
- **Transition Network (Time Slices t and t+1)**: A two-slice temporal Bayes net (2TBN) that models how variables evolve from one time step to the next. It includes:
  - **Intra-slice edges**: Dependencies within the same time slice (e.g., $X_t \rightarrow Y_t$).
  - **Inter-slice edges**: Dependencies across time slices (e.g., $X_t \rightarrow X_{t+1}$).

Handling Missing Data



Multiple imputation

Based on **Markov chain Monte Carlo (MCMC), multiple complete datasets are generated and trained separately to aggregate the results**, which can reflect missing uncertainties but has a high computational cost.

Data deletion

Directly deleting samples or variables with missing values is simple and feasible, but may result in information loss and bias. It is only applicable to the scenario of **random missing values (MCAR)**.

Model embedding method

Modeling the missing mechanism as a part of the network (such as adding hidden variables), using EM algorithm to jointly infer parameters and missing values, suitable for **complex missing patterns (MNAR)**.

MCAR vs. MAR vs. MNAR

- **MCAR: Missing Completely At Random** – the probability a value is missing does not depend on any data, observed or missing.
- **MAR: Missing At Random** – the probability a value is missing depends only on observed data, not on the unobserved value itself.
- **MNAR: Missing Not At Random** – the probability a value is missing does depend on the unobserved value itself, even after conditioning on all observed data.

Bayesian Networks Core Formula

The joint probability distribution of a Bayesian Network with variables $X_1, X_2, \ldots, X_N$ is:

$$P(X_1, X_2, \ldots, X_N) = \prod_{i=1}^{N} P(X_i | parents(X_i))$$

Key Properties:

Directed edges show direct dependencies (e.g., $A \rightarrow B$ means B depends on A).

Absence of edges implies conditional independence.

Also known as a "belief network" or "causal network"

Example 1: Simple 3-Node Network

Variables:

X_1 (Parent): $P(X_1 = true) = 0.7$

X_2 (Child of X_1): $P(X_2 = true | X_1 = true) = 0.6$

X_3 (Child of X_1): $P(X_3 = true | X_1 = true) = 0.1$

Calculate joint probability $P(X_1 = true, X_2 = true, X_3 = true)$:

$$P(X_1 = T, X_2 = T, X_3 = T) = P(X_1 = T) \cdot P(X_2 = T | X_1 = T) \cdot P(X_3 = T | X_1 = T) = 0.7 \times 0.6 \times 0.1$$
$$= 0.042$$

Example 2: Fire Alarm System

Variables:

A (Fire): $P(A) = 0.4$

B (Alarm, depends on A): $P(B|A) = 0.3$

C (Smoke, depends on B): $P(C|B) = 0.1$

D (Evacuation, depends on B): $P(D|B) = 0.95$

Calculate joint probability $P(A = T, B = T, C = T, D = T)$:

$$P(A, B, C, D) = P(A) \cdot P(B|A) \cdot P(C|B) \cdot P(D|B) = 0.4 \times 0.3 \times 0.1 \times 0.95 = 0.0114$$

Example 3: Reverse Inference (Wet Grass Problem)

Variables:

R (Rain): $P(R) = 0.3$

S (Sprinkler): $P(S|R) = \begin{cases} 0.01 & if R = T \\ 0.4 & if R = F \end{cases}$

G (Wet Grass, depends on R and S):

$$P(G|R,S) = \begin{cases} 0.99 & if R = T or S = T \\ 0.0 & otherwise \end{cases}$$

Find $P(R = T|G = T)$:

$$P(R = T|G = T) = \frac{P(G = T|R = T) \cdot P(R = T)}{P(G = T)}$$

Step 1: Compute denominator (marginalization):

$$P(G = T) = \sum_{R,S} P(G = T|R,S) \cdot P(R) \cdot P(S|R) = 0.3 \times 0.99 + 0.7 \times 0.4 \times 0.99$$

$$= 0.297 + 0.277 = 0.574$$

Step 2: Calculate posterior probability:

$$P(R = T|G = T) = \frac{0.99 \times 0.3}{0.574} \approx 0.518$$

06 Advantages and Challenges



Strengths (e.g., Prior Knowledge Integration)

Prior Knowledge Integration

Bayesian methods allow for the **incorporation of prior knowledge through prior distributions**, which can improve model performance, especially in scenarios with limited data. This is particularly useful in domains like medicine where expert knowledge is available.

Uncertainty Quantification

Bayesian approaches provide a **natural framework for quantifying uncertainty in predictions**. This is crucial for decision-making processes, as it allows practitioners to assess the confidence in model outputs and make informed choices.

Flexibility in Modeling

Bayesian models are highly flexible and can be adapted to a **wide range of complex problems**. They can handle **non-linear relationships, hierarchical structures, and missing data** more effectively than many traditional methods.

Limitations (e.g., Computational Complexity)

Computational Complexity

Bayesian methods often **require computationally intensive** techniques like **Markov Chain Monte Carlo (MCMC) for inference**, which can be slow and resource-heavy, especially for large datasets or complex models.

Choice of Priors

The performance of Bayesian models **heavily depends on the choice of prior distributions**. Poorly chosen priors can lead to biased or inaccurate results, and selecting appropriate priors often requires domain expertise.

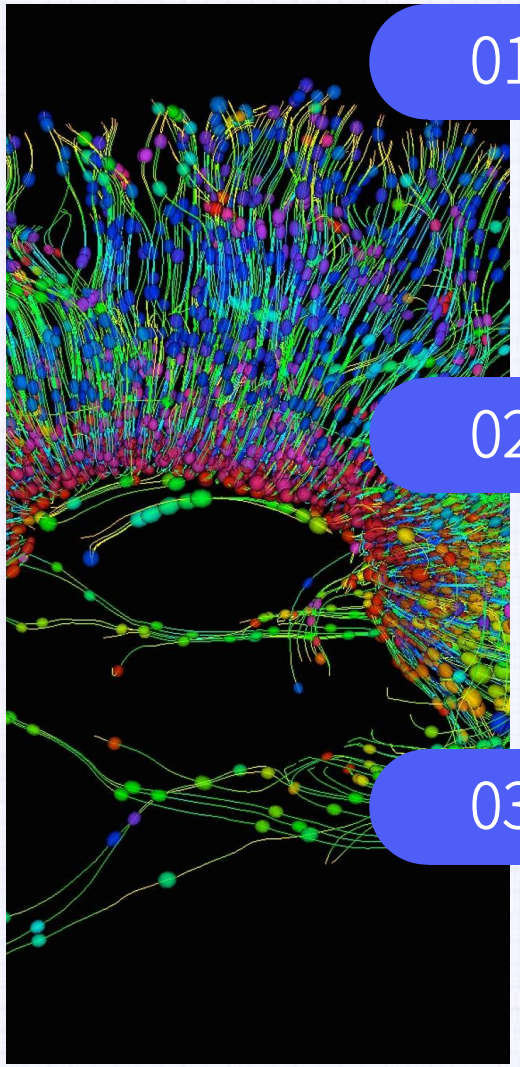
Scalability Issues

While Bayesian methods are powerful, they can **struggle with scalability when applied to very high-dimensional data or massive datasets**, limiting their use in some modern machine learning applications.

Interpretability Challenges

Although Bayesian models provide probabilistic interpretations, the results can **sometimes be difficult to interpret**, especially for non-experts, due to the complexity of posterior distributions and their interactions.

Future Research Directions



01

Scalable Inference Methods

Developing **more efficient and scalable inference techniques**, such as **variational inference** or **stochastic gradient MCMC**, to address the computational challenges of Bayesian methods in large-scale applications.

02

Automated Prior Selection

Research into **automated or data-driven approaches for selecting priors** could make Bayesian methods more accessible and reduce reliance on expert knowledge, broadening their applicability.

03

Integration with Deep Learning

Exploring ways to combine Bayesian approaches with deep learning architectures to leverage the strengths of both paradigms, such as **uncertainty-aware deep learning models for robust and interpretable predictions**.